DOCUMENTAÇÃO TÉCNICA

CATI Forms

Plataforma web de criação, compartilhamento e coleta de respostas de formulários.

Resumo da implementação de referência · RF01 a RF16 · CATI Jr.

01 / VISÃO GERAL

O CATI Forms permite que usuários cadastrados criem e consultem formulários, enquanto visitantes enviam respostas por links públicos sem necessidade de conta. As respostas podem ser consultadas e exportadas em CSV e PDF.

Acesso autenticado

Cadastro e login; criação e consulta de todos os formulários e respostas, conforme as regras de acesso do projeto.

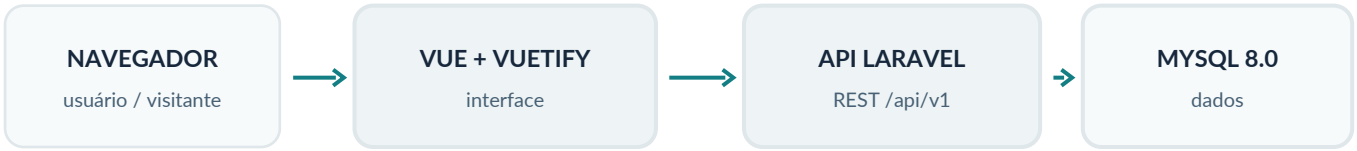
Acesso público

Preenchimento e envio por URL baseada em UUID; sem acesso aos resultados dos demais respondentes.

STACK TECNOLÓGICA

Camada	Tecnologias adotadas
Backend	PHP 8.3+, Laravel 13, Eloquent ORM e Fortify.
Frontend	Vue 3, TypeScript, Inertia.js 3 e Vuetify 3.
Navegação / API	Páginas Inertia + API REST JSON versionada em **/api/v1**.
Banco e infraestrutura	MySQL 8.0 em Docker Compose; Laravel executado no host.
Exportação	CSV via **fputcsv** (PHP); PDF via **dompdf/dompdf**.
Testes / qualidade	Pest, PHPUnit, Pint, PHPStan/Larastan e vue-tsc.

ARQUITETURA RESUMIDA



Inertia entrega páginas; Vue consulta a API; Fortify gerencia sessão; Eloquent persiste dados.

Monólito modular Laravel: as rotas web entregam as páginas via Inertia, e os formulários utilizam **fetch** para consumir endpoints JSON na mesma origem. A autenticação utiliza cookies, sessão e CSRF; não há Bearer token.

02 / FUNCIONALIDADES E DADOS

Os 16 requisitos funcionais foram implementados, agrupados abaixo pelos principais fluxos do produto.

Requisitos	Comportamento entregue
RF01–RF02	**Usuários:** cadastro com username único, autenticação e logout por sessão.
RF03–RF07	**Criação:** título e descrição; perguntas discursivas, múltipla escolha e escolha única com justificativa; configuração de coleta de e-mail.
RF08–RF09	**Compartilhamento:** link público com UUID e preenchimento sem cadastro.
RF10–RF13	**Respostas:** preenchimento obrigatório; justificativa; armazenamento transacional e associações válidas.
RF14	**Consulta:** todos os usuários autenticados visualizam formulários, detalhes e respostas.
RF15–RF16	**Exportação:** download de todas as respostas do formulário em CSV ou PDF.

MODELO DE DADOS

Entidades	Responsabilidade
users	Usuários cadastrados; podem criar formulários.
forms	Título, descrição, UUID público, criador e configuração de e-mail.
questions / question_options	Perguntas de cada formulário e alternativas das questões objetivas.
submissions	Um envio vinculado ao formulário; e-mail opcional.
answers / answer_options	Respostas por pergunta e alternativas efetivamente marcadas.

Relações principais: **formulário → perguntas → alternativas** e **formulário → submissões → respostas → alternativas selecionadas**. As entidades são relacionadas por chaves estrangeiras; a aplicação valida se os identificadores enviados pertencem ao formulário correto.

INTEGRIDADE E REGRAS DE NEGÓCIO

- Formulários são gravados com perguntas e alternativas em **transação**; respostas e marcações também são salvas atomicamente.
- Todas as perguntas são obrigatórias no envio. Escolha única exige **uma alternativa e justificativa**; múltipla escolha aceita várias marcações válidas.
- Formulários não oferecem edição ou exclusão após a criação. Usuários cadastrados têm acesso equivalente aos formulários e resultados.

03 / API REST E SEGURANÇA

A API de formulários usa o prefixo **/api/v1**. Os endpoints administrativos exigem sessão autenticada; os públicos usam o UUID do formulário.

HTTP	Endpoint	Acesso	Finalidade
POST	/forms	Privado	Criar formulário
GET	/forms; /forms/mine	Privado	Listar todos / meus
GET	/forms/{id}	Privado	Detalhes do formulário
GET	/forms/{id}/submissions	Privado	Consultar respostas
GET	/forms/{id}/exports/csv	Privado	Baixar CSV
GET	/forms/{id}/exports/pdf	Privado	Baixar PDF
GET	/public/forms/{uuid}	Público	Acessar formulário
POST	/public/forms/{uuid}/submissions	Público	Enviar respostas

CONTRATO E TRATAMENTO DE ERROS

As respostas JSON usam recursos delimitados; listagens têm paginação de 12 itens. Um envio público válido retorna confirmação sem revelar respostas ou IDs de submissão. Códigos relevantes: **401** (não autenticado), **404** (não encontrado), **419** (CSRF), **422** (validação) e **429** (limite de requisições).

SEGURANÇA IMPLEMENTADA

- **Autenticação:** Fortify, sessão e cookies; senhas armazenadas como hash e username único.
- **Proteção de requisições:** middleware **auth:web** em recursos privados, CSRF em POSTs e validação por Form Requests.
- **Abusos:** limite de 5 tentativas/minuto no login por username/IP e 60 envios públicos/minuto por IP.
- **Integridade:** verificação de perguntas/alternativas e transações nas gravações; consultas parametrizadas via Eloquent.
- **Saída segura:** CSV protege células iniciadas como fórmulas; PDF desativa scripts, PHP e acesso remoto no Dompdf.
- **Dados sensíveis:** respostas e exports não ficam nos endpoints públicos; **.env** e **.env.docker** são arquivos locais ignorados.

EXPORTAÇÃO

CSV: UTF-8, uma linha por submissão, colunas por pergunta e justificativa adicional na escolha única. **PDF:** A4, organização por submissão, perguntas e respostas, com fonte compatível com acentos. Ambos são downloads privados, sem retorno JSON no sucesso.

04 / EXECUÇÃO E VALIDAÇÃO

AMBIENTE LOCAL (RESUMO)

O procedimento completo está no **README.md** do repositório. O Laravel executa no host e acessa o MySQL do Docker pela porta local configurada (exemplo do projeto: **3308**).

```
cp .env.example .env
cp .env.docker.example .env.docker
docker compose --env-file .env.docker up -d db
composer install   # instalar dependências PHP
npm ci --include=optional
php artisan key:generate
php artisan migrate
npm run build && make dev
```

Antes de executar as migrations, alinhe **DB_HOST=127.0.0.1**, **DB_PORT**, nome do banco, usuário e senha do **.env** com o **.env.docker**. O container cria o banco na inicialização de um volume vazio; as migrations criam as tabelas.

TESTES E QUALIDADE

Verificação	Evidência registrada
Suíte automatizada	**271 testes aprovados** e **1.642 asserções** no registro final da implementação.
Isolamento	Pest/Feature usam SQLite **:memory:** com proteções contra execução em banco persistente.
Qualidade	PHPStan, verificação de tipos Vue/TypeScript, Pint/lint e build registrados como aprovados.
Cobertura de cenários	Autenticação, validações, CSRF, persistência, consultas, exportações CSV/PDF e erros.

Resultados extraídos da documentação técnica fornecida. Não houve nova execução da suíte para gerar este PDF.

LIMITAÇÕES E EVOLUÇÃO

- Fora do escopo atual: edição/exclusão de formulários, perfis de acesso, perguntas opcionais e tipos adicionais.
- A API atual usa sessão na mesma origem; integração com sistemas externos e o módulo de Controle de Pessoas ainda não foram implementados.
- Exportação PDF em grande volume depende de memória; não há processamento assíncrono de exportações.
- Implantação de produção requer revisão de HTTPS, backups, configuração de debug e infraestrutura.

REFERÊNCIA

Documento-fonte: documentacao-tecnica.md (revisão 11/11/2026; base 63f896e).
Contratos REST e roteiro Postman: páginas 5-10. Instalação completa: README.md.

05 / API REST: CONTRATOS E ACESSOS

O módulo de formulários expõe endpoints JSON em **/api/v1**, na mesma origem da aplicação. Cadastro, login e logout utilizam rotas web do Laravel Fortify, fora desse prefixo.

AUTENTICAÇÃO E HEADERS

- **Rotas privadas:** exigem sessão autenticada via **auth:web**; não há Bearer token nem `/sanctum/csrf-cookie`.
- **Requisições JSON:** enviar `Accept: application/json`; nos POSTs, enviar também `Content-Type: application/json` e `X-XSRF-TOKEN`.
- **Cookies:** manter `XSRF-TOKEN` e cookie de sessão no mesmo host. O POST público também exige CSRF, mesmo sem login.

ENDPOINTS DISPONÍVEIS

HTTP	Rota (após /api/v1)	Acesso / sucesso
POST	/forms	Sessão + CSRF · 201
GET	/forms	Sessão · 200, todos
GET	/forms/mine	Sessão · 200, meus
GET	/forms/{form}	Sessão · 200, detalhes
GET	/forms/{form}/submissions	Sessão · 200, respostas
GET	/forms/{form}/exports/csv	Sessão · 200, CSV
GET	/forms/{form}/exports/pdf	Sessão · 200, PDF
GET	/public/forms/{public_id}	Público · 200
POST	/public/forms/{public_id}/submissions	Público + CSRF · 201

Identificadores: `{form}` é o ID numérico interno; `{public_id}` é o UUID público. A página aberta no navegador é `/f/{public_id}`.

FORMATO DAS RESPOSTAS

Detalhes usam envelope `data`. Listagens retornam `data`, `links` e `meta`, com **12 itens por página** e parâmetro `?page=1`. Perguntas e alternativas vêm nos detalhes; as listagens de formulários não as incluem.

06 / API REST: CRIAR E CONSULTAR

CRIAR UM FORMULÁRIO | POST /api/v1/forms

Requer usuário autenticado e CSRF. O servidor define UUID, autoria, identificadores e posições; a gravação de formulário, perguntas e alternativas é transacional.

```
{
  "title": "Pesquisa CATI",
  "description": "Compartilhe sua opinião.",
  "request_contact_email": true,
  "questions": [
    { "type": "discursive", "prompt": "Como melhorar?" },
    { "type": "multiple_choice", "prompt": "Quais áreas interessam?",
      "options": [{ "label": "Desenvolvimento" }, { "label": "Design" }] },
    { "type": "single_choice_with_justification",
      "prompt": "Qual formato prefere?",
      "options": [{ "label": "Online" }, { "label": "Presencial" }] }
  ]
}
```

Campo	Regra documentada
title	Obrigatório; string de até 255 caracteres.
description	Obrigatória; até 10.000 caracteres.
request_contact_email	Booleano opcional; padrão false.
questions	Lista opcional; [] permite formulário sem perguntas.
questions.N.type	discursive, multiple_choice ou single_choice_with_justification.
questions.N.prompt	Obrigatório; até 10.000 caracteres.
questions.N.options	Obrigatória nas objetivas (mínimo 1); ausente ou [] na discursiva.
questions.N.options.N.label	Obrigatório; até 10.000 caracteres.

Sucesso: HTTP **201** com `data` contendo formulário, UUID, URL pública, perguntas e alternativas. Um novo POST válido cria outro formulário; não há edição ou exclusão posterior.

CONSULTAS PRIVADAS

`GET /forms` lista todos; `GET /forms/mine` filtra pelo criador; `GET /forms/{form}` retorna perguntas; `GET /forms/{form}/submissions` retorna envios e respostas. Qualquer usuário autenticado pode consultar qualquer formulário (RN02/RN10).

07 / API REST: RESPOSTAS E ARQUIVOS

CONSULTA PÚBLICA | GET /api/v1/public/forms/{public_id}

Retorna título, descrição, opção de e-mail e perguntas com alternativas, sem revelar respostas anteriores, autoria ou e-mails coletados. O acesso é anônimo e utiliza UUID.

ENVIO PÚBLICO | POST /api/v1/public/forms/{public_id}/submissions

```
{
  "contact_email": "participante@example.com",
  "answers": [
    { "question_id": 1, "text": "Mais encontros técnicos." },
    { "question_id": 2, "option_ids": [4, 5] },
    { "question_id": 3, "option_ids": [8],
      "justification": "Facilita a participação." }
  ]
}
```

Importante: os IDs acima são ilustrativos. O envio deve usar os IDs reais retornados ao criar ou consultar o formulário.

- **Todas as perguntas:** uma resposta por pergunta, sem repetições e com IDs pertencentes ao formulário.
- **Discursiva:** texto não vazio; **múltipla escolha:** uma ou mais alternativas distintas; **escolha única:** exatamente uma alternativa e justificativa não vazia.
- **E-mail:** continua opcional mesmo quando solicitado; quando preenchido, precisa ser válido. Se a coleta está desativada, e-mail não vazio é rejeitado.

```
HTTP/1.1 201 Created
{ "message": "Resposta enviada com sucesso." }
```

Consistência: submissão, respostas e alternativas selecionadas são gravadas em transação. Se a gravação falhar, o conjunto é revertido. Repetir um POST válido cria outro envio.

EXPORTAÇÕES | GET AUTENTICADO

Formato / rota	Retorno
`/forms/{form}/exports/csv`	200 · text/csv; charset=UTF-8 · uma linha por submissão
`/forms/{form}/exports/pdf`	200 · application/pdf · A4 por submissões

Os dois downloads usam `Content-Disposition: attachment`, incluem **todos os envios** e não retornam JSON em sucesso. Sem respostas: CSV com cabeçalho e PDF com mensagem de ausência.

STATUS HTTP IMPORTANTES

401 sem sessão · **404** formulário ausente/UUID inválido · **419** CSRF inválido · **422** validação · **429** limite de requisições · **500** erro do servidor. Erro 422 inclui `message` e `errors`.

08 / POSTMAN: CONFIGURAÇÃO E CSRF

Este roteiro usa o contrato real documentado. Os exemplos **criam registros**: use um ambiente local de desenvolvimento. Não utilize credenciais reais nem exporte cookies de sessão.

CRIAR UMA COLEÇÃO

- Defina a autenticação da coleção como **No Auth** e mantenha o **Cookie Jar** habilitado.

Variável	Valor inicial
base_url	http://localhost:8000 (sem / final)
username	postman_teste_01 (um nome novo)
password	Senha local de teste, forte (>= 12 caracteres)

Em todas as requisições de dados, configure `Accept: application/json`. Nos POSTs com corpo, escolha **Body → raw → JSON** e `Content-Type: application/json`. Use sempre o mesmo host: não alterne `localhost` e `127.0.0.1`.

SCRIPT DE PRÉ-REQUISIÇÃO (NA COLEÇÃO)

Em **Scripts → Pre-request**, adicione o script documentado para obter o cookie XSRF-TOKEN e enviá-lo nos POSTs. O token pode mudar depois do login e logout.

```
if (['POST', 'PUT', 'PATCH', 'DELETE'].includes(pm.request.method)) {
    const base = pm.collectionVariables.get('base_url');
    const publicId = pm.collectionVariables.get('public_id');
    const path = pm.request.url.toString();
    const csrfUrl = path.includes('/public/forms/') && publicId
        ? `${base}/api/v1/public/forms/${publicId}`
        : `${base}/login`;

    pm.sendRequest({
        url: csrfUrl,
        method: 'GET',
        header: { Accept: 'application/json' },
    }, (error, response) => {
        if (error) throw error;
        const cookie = response.cookies.get('XSRF-TOKEN');
        if (!cookie) throw new Error('Sem XSRF-TOKEN; confira Cookie Jar.');
        pm.request.headers.upsert({
            key: 'X-XSRF-TOKEN',
            value: decodeURIComponent(cookie),
        });
    });
}
```

Atenção: o GET preparatório obtém o token no mesmo contexto de cookies. Se houver erro 419, verifique primeiro cookies, host, header X-XSRF-TOKEN e o token da sessão atual. Não use Bearer token.

09 / POSTMAN: AUTENTICAR E CRIAR

PASSO 1 | CADASTRO OU LOGIN

Cadastro: `POST {{base_url}}/register` (retorna **201** e cria sessão). Para conta existente: `POST {{base_url}}/login` (retorna **200**). Ambos exigem CSRF.

```
// Cadastro - body raw JSON
{
  "username": "{{username}}",
  "password": "{{password}}",
  "password_confirmation": "{{password}}"
}

// Login de conta existente - body raw JSON
{
  "username": "{{username}}",
  "password": "{{password}}",
  "remember": false
}
```

Depois do login, consulte `GET {{base_url}}/api/v1/forms/mine`. Esperado: **200**. Credenciais inválidas retornam **422**.

PASSO 2 | CRIAR FORMULÁRIO

Use `POST {{base_url}}/api/v1/forms` com o JSON completo da página 6. Esperado: **201**. A resposta inclui os identificadores reais das perguntas e alternativas.

PASSO 3 | SALVAR IDS AUTOMATICAMENTE

Na requisição de criação, coloque em **Scripts → Post-response** o script abaixo. Ele pressupõe a ordem dos três tipos no exemplo da página 6.

```
pm.test('Formulário criado', () => pm.response.to.have.status(201));
if (pm.response.code === 201) {
    const form = pm.response.json().data;
    pm.collectionVariables.set('form_id', form.id);
    pm.collectionVariables.set('public_id', form.public_id);
    pm.collectionVariables.set('public_url', form.public_url);
    const [discursive, multiple, single] = form.questions;
    pm.collectionVariables.set('answers', JSON.stringify([
      { question_id: discursive.id,
        text: 'Mais encontros técnicos. Sugestão com acentos.' },
      { question_id: multiple.id,
        option_ids: multiple.options.map(option => option.id) },
      { question_id: single.id,
        option_ids: [single.options[0].id],
        justification: 'Facilita a participação.' },
    ]));
}
```

Conferência: use `GET /api/v1/forms`, `GET /api/v1/forms/mine` e `GET /api/v1/forms/{{form_id}}` (todos privados). Listagens têm 12 itens por página.

10 / POSTMAN: ENVIO, CONSULTA E DOWNLOAD

PASSO 4 | RESPONDER SEM ESTAR LOGADO

Faça `POST {{base_url}}/logout` (esperado **204**) ou utilize outra sessão sem autenticação. A consulta privada passa a retornar **401**. Faça então:

```
GET {{base_url}}/api/v1/public/forms/{{public_id}}

// Esperado: 200. Retorna definição do formulário,
// sem dados de outros respondentes.
```

Para enviar, use **Body → raw → JSON**. A variável `{{answers}}` foi gerada no passo anterior e deve ficar **sem aspas**, pois já é uma lista JSON.

```
POST {{base_url}}/api/v1/public/forms/{{public_id}}/submissions

{
  "contact_email": "participante@example.com",
  "answers": {{answers}}
}

// HTTP 201
{ "message": "Resposta enviada com sucesso." }
```

Se a coleta de e-mail estiver desativada, omita `contact_email`. Repetir o POST cria **outra submissão**; não reenvie automaticamente em caso de falha de rede.

PASSO 5 | CONFERIR AS RESPOSTAS

Autentique-se novamente e consulte `GET {{base_url}}/api/v1/forms/{{form_id}}/submissions`. Confira texto, alternativas, justificativa e e-mail da submissão criada. O endpoint público não pode devolver esses dados.

PASSO 6 | BAIXAR AS EXPORTAÇÕES

```
GET {{base_url}}/api/v1/forms/{{form_id}}/exports/csv
GET {{base_url}}/api/v1/forms/{{form_id}}/exports/pdf
```

Use **Send and Download** (ou **Save Response → Save to a file**). Salve com extensão .csv/.pdf e confira **200**, `Content-Disposition: attachment` e o conteúdo exportado. Não espere JSON de sucesso.

CASOS INVÁLIDOS ESSENCIAIS

Situação testada	Resposta esperada
Consulta privada após logout	401 JSON
UUID público inexistente	404 JSON
Título ou justificativa ausente	422 com errors.title ou errors.*.justification
Pergunta/alternativa de outro formulário	422 no campo de pergunta/alternativa
POST sem token CSRF válido	419
Limite de envios públicos excedido	429 + Retry-After (não forçar em ambiente compartilhado)

Fonte dos contratos e exemplos: **forms.md** e **postman.md** fornecidos com a documentação. Estes exemplos descrevem o comportamento documentado; não representam uma nova execução dos testes no Postman.